

Sala de Estudo Inteligente: Implementação de uma Plataforma de Sensorização Multimodal para Bibliotecas Universitárias

Luís Silva¹, Pedro Reis¹, Guilherme Pinto¹, and Pedro Gomes¹

Universidade do Minho, Departamento de Informática,
Mestrado em Inteligência Artificial
{pg60390, pg59908, pg60225, pg60289}@alunos.uminho.pt

Resumo Em sequência ao levantamento do estado da arte produzido na primeira fase do trabalho prático, este artigo apresenta a implementação e validação de um *protótipo funcional* de baixo custo para detecção de ocupação e monitorização do conforto ambiental em salas de estudo universitárias. O protótipo assenta em dois nós de sensorização baseados em ESP32 — um nó de visão (ESP32-CAM com câmara OV2640) e um nó ambiental (ESP32-S3 com DHT11, MQ-135, fotodíodo LM393 e microfone MEMS I²S) — ligados a uma infraestrutura *cloud* em Firebase organizada em dois projetos isolados, que alimenta um *pipeline* de inferência YOLOv11x com descoberta automática do *layout* (mesas) e atribuição de deteções por proximidade, incluindo casos de *seat hogging*. Os dados são expostos por uma API REST consumida por um *dashboard* web multi-sala e por uma aplicação Android nativa com *geofencing*, alertas configuráveis e catálogo bibliográfico. Acresce um módulo preditivo que compara três abordagens (média horária, Holt-Winters, LSTM) para previsão de séries ambientais. Para além da implementação concreta, o artigo formula o protótipo como o *cenário A* de uma família de três estratégias de implantação por sala — *cloud*, centralizado *on-premise* e *edge* dedicado — e analisa-as nos eixos de *custo* (CAPEX, OPEX, *licenciamento de software*), *privacidade*, *granularidade* e *escala*. A análise crítica delimita honestamente as limitações actuais do protótipo — nomeadamente a precisão de sensores analógicos de baixo custo, a janela curta de dados reais disponível para validar os modelos preditivos, e a sensibilidade do YOLO ao ângulo de captura e à iluminação.

Keywords: IoT · Visão Computacional · YOLO · ESP32 · Conforto Ambiental · Inteligência Ambiente · Firebase

1 Introdução

As bibliotecas universitárias são espaços de utilização intensiva e procura desigual ao longo do dia: em períodos de exames, alunos percorrem vários pisos em busca de um lugar livre, perdendo tempo de estudo efetivo. O problema consiste na prática, de dois subproblemas: tempo desperdiçado em deslocações entre pisos e estudo em ambientes com conforto deficiente. A ineficiência é agravada

por dois fenómenos paralelos que sensores não-visuais não distinguem: mesas vazias mas reservadas por objetos pessoais (*seat hogging*) e variação de conforto ambiental (temperatura, ar, ruído, luz) que afasta utilizadores mesmo quando há lugares disponíveis. A disponibilização em tempo real de informação sobre ambas as dimensões reduz significativamente o tempo médio de procura de um lugar e permite escolher a sala com melhores condições para estudo [1].

O presente artigo descreve uma plataforma de três camadas (sensorização → processamento → apresentação) que ataca este problema. A arquitetura assenta em Firebase com dois projetos isolados, separando fisicamente imagens e dados ambientais — escolha de *privacy-by-design* alinhada com o RGPD.

A contribuição é dupla: um protótipo operacional em ESP32 com BOM (*Bill of Materials* — custo agregado dos componentes de *hardware*) inferior a 65 €/sala, replicável por estudantes e investigadores; e arquitetura modular onde cada bloco (aquisição visual, inferência, sensorização ambiental, API) pode ser substituído sem quebrar o contrato de dados. A Secção 2 descreve o protótipo e a Secção 5.2 formaliza-o como *cenário A* de três implementações possíveis, com *trade-offs* em *hardware*, *software* (licenciamento incluído), privacidade e escala.

2 Arquitetura do Sistema e Sensorização

2.1 Arquitetura Geral e Fluxo de Dados

A Figura 1 sintetiza as três camadas da arquitetura implementada. O nó de visão captura imagens periodicamente e envia-as para o *bucket* Firebase Storage do projeto VISION; o nó ambiental publica leituras a cada 30 s no Realtime Database do projeto SENSOR. Um processo no servidor descarrega cada nova imagem, executa a inferência YOLO e a atribuição por mesa, agrega o resultado e escreve o estado de ocupação no projeto SENSOR, fechando a integração. Uma API REST em Flask agrega os dados de ambos os domínios num contrato JSON único, consumido pelo *frontend* web e pela aplicação Android. A escolha de duas *apps* Firebase distintas para imagens e dados ambientais garante que credenciais, regras de acesso e quotas das imagens nunca se misturam com a telemetria de conforto.

A arquitetura aqui descrita corresponde ao *cenário A* da análise de implantação (Secção 5.2): um protótipo de prova-de-conceito otimizado para baixo CAPEX de validação. Os cenários B e C substituem blocos específicos — o nó de visão e o local da inferência — mantendo invariantes o nó ambiental, o contrato JSON da API e a camada de apresentação.

2.2 Nó Visão

Hardware e desafios práticos. O nó de visão assenta no módulo ESP32-CAM AI-THINKER com SoC ESP32-S dual-core a 240 MHz, 4 MB PSRAM, sensor OV2640 de 2 MP, $\approx 27 \times 40$ mm, e <10 €. A programação envolveu três desafios: (i) ausência de USB nativo, resolvida com adaptador YP-05 e ponte

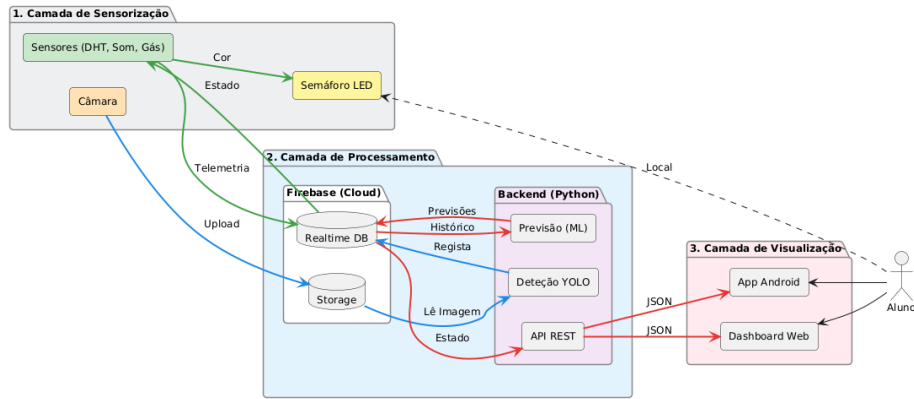


Figura 1. Arquitetura do sistema em três camadas.

IO0→GND para modo de *flashing* (Figura 2); (ii) consumo em pico (~ 300 mA) causava *brownout*, resolvido com fonte externa 5 V/ ≥ 1 A e desativação por *software* do LED *flash*; (iii) resolução UXGA (1600×1200) saturava o Wi-Fi (60–90 s/*upload*), substituída por SVGA (800×600 , 5–10 s) sem perda relevante, pois o YOLO redimensiona internamente para 640×640 .

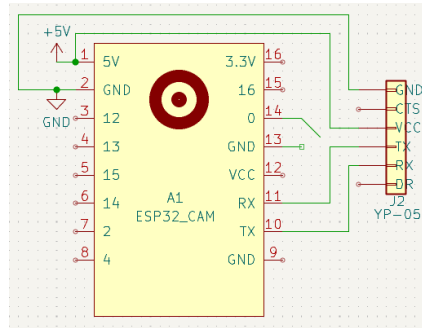


Figura 2. Esquema elétrico do nó de visão.

Programação e papel como sensor visual. O desenvolvimento usou PLATFORMIO pela gestão declarativa de dependências. O *firmware* (C++/Arduino) configura os 16 pinos da OV2640 via BSP oficial, declara JPEG com XCLK de 20 MHz e duplo *frame buffer* em PSRAM para capturas não-bloqueantes; a biblioteca Firebase com stack TLS ultrapassa 1.2 MB, exigindo a reconfiguração da tabela de partições para alocar uma partição de aplicação alargada, sem suporte a atualizações remotas de *firmware*. A cada intervalo, envia o JPEG ao Storage

e escreve o caminho no Realtime Database — mecanismo de sinalização que evita *polling* direto ao Storage. A ESP32-CAM é o coletor crítico do sistema: fornece a informação visual sem a qual nenhuma abordagem não-visual (PIR, WiFi-sensing, ultrassom) consegue distinguir mesas vazias de mesas reservadas por objetos pessoais (*seat hogging*) [1]. Após inferência, a imagem é apagada tanto localmente como do Firebase Storage, nunca sendo persistida.

2.3 Nó Ambiental

Plataforma. O nó ambiental usa um ESP32-S3 DevKitC-1 em vez da ESP32-CAM por dispor de mais pinos ADC e GPIO livres, integrar USB-CDC nativo e ter um controlador PWM mais flexível para os LEDs do semáforo. A Figura 3 mostra o esquema completo: DHT11 em 1-wire (GPIO 4), MQ-135 via divisor 2k/2k em ADC (GPIO 5), fotodíodo LM393 com saídas analógica e digital (GPIO 6 e 7), microfone MEMS MSM261 em I²S (GPIO 13, 14 e 15) e três LEDs PWM (GPIO 16, 17 e 18).

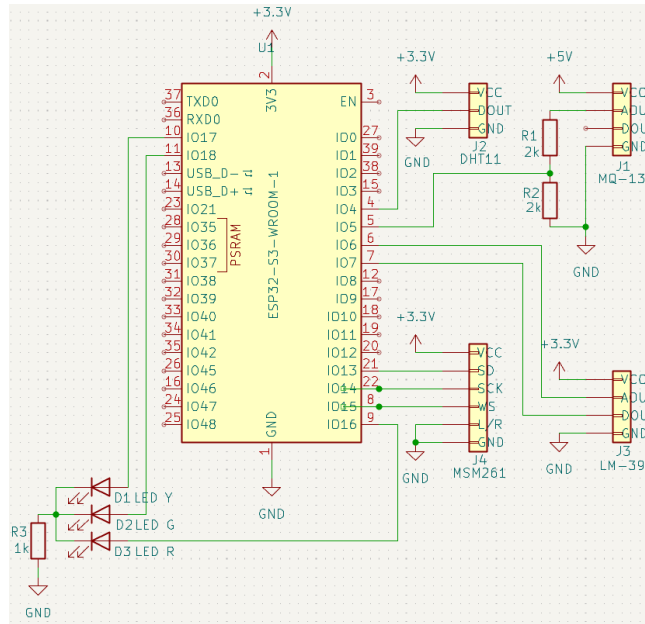


Figura 3. Esquema elétrico do nó ambiental.

Sensores, aquisição e schema de envio. **Temperatura e humidade** são fornecidas pelo DHT11 via biblioteca 1-wire dedicada, com leitura digital a cada 30 s. **Qualidade do ar** é estimada pelo MQ-135, sensor de SnO₂ que entrega

uma tensão analógica proporcional à concentração agregada de gases redutores (CO_2 , NH_3 , NO_x). Por exigir alimentação a 5 V, a sua saída pode atingir tensões acima do limite ADC do ESP32-S3 (3.3 V); resolvemos esta incompatibilidade com um divisor resistivo $R_1=R_2=2\text{ k}\Omega$: a tensão no GPIO é $V_{\text{GPIO}} = V_{\text{out}} \cdot R_2/(R_1+R_2) = V_{\text{out}}/2$, garantindo $V_{\text{GPIO}} \leq 2.5\text{ V}$ em pior caso (saturação do sensor a 5 V), o que utiliza cerca de 76% da gama do conversor de 12 bits do ESP32-S3. Esta margem é desejável: ficar abaixo do limite superior do ADC evita não-linearidades conhecidas do conversor da família ESP32 junto ao topo da gama. **Iluminância** é obtida com um módulo fotodíodo baseado no comparador LM393 (saída analógica de intensidade relativa e digital com limiar pré-calibrado por potenciômetro), usadas em conjunto para classificar a iluminação em quatro classes qualitativas (*escuro*, *baixa*, *adequada*, *elevada*). **Ruído** é medido por um microfone MEMS digital MSM261S4030H0 interfaceado por I²S a 16 kHz; o *firmware* lê janelas de 1024 amostras de 24 bits, calcula o RMS e converte para dB conforme $\text{dB} = 20 \log_{10}(\text{RMS}/2^{23}) + 94$, onde a constante +94 dB é a aproximação SPL na qual a sensibilidade nominal do MSM261 (−26 dBFS @ 94 dB SPL @ 1 kHz) é definida no *datasheet*; o valor calculado é indicativo e não metrológico, em linha com sistemas IoT comparáveis para monitorização acústica em ambientes educativos [1].

Em vez de um LED RGB único, a montagem evoluiu para três LEDs discretos (vermelho, amarelo, verde) controlados por PWM, com maior visibilidade à distância e analogia direta com um semáforo. O nó subscreve o estado de ocupação publicado pelo *pipeline* de visão e consulta o Firebase a cada 5 s, sem recalculá-lo localmente — a latência efetiva do semáforo é, porém, dominada pelo ciclo de captura e inferência (~30 s ponta-a-ponta), garantindo ainda assim consistência com os clientes web/Android. O *firmware* agrega num único objeto JSON 11 métricas (5 numéricas brutas, 4 classes textuais, conforto agregado e *timestamp*) e usa duas escritas atômicas no Realtime Database: uma substitutiva no *snapshot* corrente e outra acumulativa no histórico.

Limiares e norma. As classificações textuais usam limiares ancorados em normas onde existem dados de calibração defensáveis (ASHRAE 55 para temperatura [3], EN 12464-1 para iluminância [4], OMS para ruído [5]). Os limiares do MQ-135 são, em rigor, calibração empírica do protótipo (≤ 800 de 4095 considerado “aceitável”), não traçável a uma norma de qualidade do ar interior — limitação retomada na Secção 6.

3 Exploração, Tratamento e Visualização de Dados

3.1 Pipeline, Armazenamento e Limpeza

Os dados percorrem três estágios. (i) No coletor são aplicadas classificações textuais baseadas nos limiares descritos — decisão deliberada: reduz o *payload* transmitido e isola a lógica de conforto da camada *cloud*, permitindo evoluir o sistema sem atualizar o *firmware* dos ESP32. (ii) No processamento, o serviço Python que consome as imagens computa a ocupação por mesa (Secção 4.1)

e publica três artefactos no Firebase: o *snapshot* de ocupação, o histórico e o estado simplificado (três níveis para o semáforo). (iii) A API REST realiza a fusão num *schema* JSON único, recalculando a classificação UX em cinco níveis por limiar de ocupação — *vazio* (0%), *disponível* (<40%), *parcialmente ocupado* (40–75%), *quase cheio* (75–95%), *cheio* ($\geq 95\%$).

A escolha do Firebase Realtime Database [6] em detrimento de uma série temporal dedicada baseou-se na latência baixa em escritas frequentes, no suporte nativo a autenticação por conta de serviço para o ESP32 e no custo nulo no nível gratuito. A limpeza acontece em três pontos: no nó ambiental, leituras inválidas (NaN, frequentes após *reset*) são substituídas por -1 ; no serviço de previsão, a série é reamostrada para 1 ponto/min com interpolação de lacunas até 10 min; e o exportador para ML alinha por *timestamp* as séries ambiental (30 s) e de ocupação (15 s a ~ 2 min), produzindo um CSV pronto para treino.

3.2 Visualização

A camada de apresentação tem três faces. O dashboard web (HTML + JavaScript) lista as sete bibliotecas do parque universitário da UMinho, identificando claramente quais estão sensorizadas. A página de detalhe de uma biblioteca faz *polling* a cada 15 s a um *endpoint* da API e renderiza uma planta esquemática em que cada mesa monitorizada (descoberta automaticamente, Secção 4.1) é pintada de verde (livre), amarelo (parcial, incluindo *seat hogging*) ou vermelho (cheia); cinco *tiles* de sensores apresentam valor, classe textual e tendência, e um gráfico Chart.js sobrepõe histórico e previsão de 1 hora, expondo o modelo preditivo ativo a cada momento.

A aplicação Android nativa (Kotlin) espelha o *frontend* web e acrescenta três funcionalidades móveis: *geofencing* de 150 m centrado na biblioteca (notificação à entrada e saída com a ocupação atual), agendador periódico que verifica condições configuráveis pelo utilizador (ocupação, conforto) e notifica quando alguma se cumpre, e um catálogo de livros que demonstra a integração com serviços de biblioteca.

Por fim, o semáforo de três LED's no nó ambiental é a manifestação mais imediata da informação, decidida globalmente no servidor para garantir consistência com os restantes clientes.

4 Modelação e Machine Learning

4.1 Inferência Visual

Escolha do modelo. O estado da arte da primeira fase apontava para o YOLOv8 como o melhor compromisso entre maturidade e precisão [1]. Reavaliámos a decisão com o lançamento do YOLOv11 [2]: o *checkpoint* extra-large (YOLOv11x; ~ 109 MB, 56.9 M parâmetros) apresentou melhor recuperação em ângulo picado — condição operacional do protótipo, em que pessoas surgem em perspectiva menos representada nos *datasets* tradicionais (COCO) do que a vista frontal canónica. Um exemplo qualitativo da saída é apresentado no Anexo A.

Classes ativas e papéis. A inferência usa um conjunto restrito de classes COCO distribuídas por três papéis. Os ocupadores — pessoa, mochila, mala de mão, garrafa, portátil e livro — contribuem para a contagem de lugares ocupados. O mobiliário (cadeira e mesa) é usado para descoberta automática do *layout* da sala e como anotação visual em *frames* de *debug*. Os distratores (telemóvel, rato, teclado, copo e TV) são incluídos na inferência para que o YOLO não force objetos pequenos retangulares a telemóvel por falta de alternativa, mas são explicitamente excluídos da contagem — incluindo o telemóvel, que classificámos como distrator (e não ocupador) com *threshold* elevado: no ângulo de captura do protótipo, um estojo ou carregador é fisicamente indistinguível de um telemóvel, e o sinal de um telemóvel sozinho na mesa é fraco uma vez que a maioria dos estudantes leva o seu.

Os *thresholds* de confiança são configurados por classe: 0.40 para pessoa, ajustado de 0.55 após observarmos que segundas pessoas parcialmente tapadas no ângulo de captura do protótipo saem com confiança 0.40–0.60 (Figura 5); 0.35–0.45 para objetos pequenos e 0.10–0.15 para a mobília, só consumida em descoberta e anotação. Limites de tamanho de *bounding-box* protegem contra pessoas em cartazes e mobília inteira marcada como portátil; uma deduplicação por $\text{IoU} \geq 0.55$ entre *boxes* de pessoa elimina duplicados em casos de oclusão parcial.

Descoberta automática do layout. Numa câmara estática sobre uma sala de estudo universitária — onde a disposição do mobiliário é fixa e não pode ser alterada pelos utilizadores —, o conjunto de mesas e cadeiras visíveis é constante. Aproveitámos esta propriedade para evitar a calibração manual de coordenadas: no primeiro arranque, sem *layout* hardcoded, o detector acumula 5 *frames* com sala assumida vazia e executa uma fase dedicada sobre o mobiliário. Deteções de mesa próximas ($\text{IoU} \geq 0.45$) são consolidadas como uma única entidade, cadeiras são atribuídas à mesa mais próxima, e o *layout* resultante (mesas com centróide normalizado e capacidade igual ao número de cadeiras detetadas) é persistido no Firebase. Como a câmara é estática, esta descoberta corre uma única vez por instalação. Em caso de falha (*e.g.* mesa muito tapada), é possível sobrepor manualmente as posições das mesas via configuração.

Atribuição por mesa e contagem. Em *runtime*, cada deteção de ocupador é atribuída à mesa mais próxima do seu *bottom-center*, com um limiar de distância máxima que descarta pessoas em circulação fora do campo útil. Para cada mesa m com capacidade cap_m , a contagem segue três regras de mútua exclusão:

- **Pessoas presentes** ($p_m > 0$): $\text{ocupados}_m = \min(p_m, \text{cap}_m)$; estado *cheio* se $\text{ocupados}_m \geq \text{cap}_m$, caso contrário *parcial*.
- **Apenas objetos** ($p_m = 0$, $o_m > 0$): $\text{ocupados}_m = \min(o_m, \text{cap}_m)$; estado *parcial* (**seat hogging** — mesa reservada por objetos na ausência de pessoas).
- **Vazia**: $\text{ocupados}_m = 0$; estado *livre*.

O estado global da sala é a soma sobre as mesas, com a percentagem de ocupação a alimentar o LED (3 níveis) e o estado UX para a planta (5 níveis). A heurística por mesa é estável e expõe uma propriedade útil ao *frontend*: a planta pode pintar cada mesa individualmente.

Granularidade ao lugar individual — decisão de âmbito. A contagem por mesa reporta o número de cadeiras ocupadas, *e.g.* 2 de 4, pelo que o utilizador sabe quantos lugares estão livres em cada mesa. A atribuição à cadeira individual foi explorada e não foi integrada: a variabilidade do YOLO na deteção de cadeiras parcialmente sob mesas produzia estados instáveis, ao passo que a contagem por mesa era estável. Identificar *qual* cadeira específica está ocupada fica para evolução futura, com câmaras em perspetiva mais propícia (Secção 5.2).

Rastreamento temporal — decisão revista. DEEPSORT, proposto na primeira fase, não foi integrado: o intervalo de captura efetivo da ESP32-CAM ($\approx 15\text{--}30$ s) é demasiado esparso para a associação por aparência agregar valor; usamos contagem direta por *frame*.

4.2 Modelos Preditivos

O componente preditivo compara três métodos para previsão a curto prazo de variáveis ambientais ($t + 60$ min):

1. **Baseline horária:** média histórica por hora-do-dia. Surpreendentemente forte em sinais periódicos como temperatura interior, estabelece o “teto mínimo” a superar.
2. **Holt-Winters** aditivo com período sazonal diário, requer ≥ 2 dias de histórico.
3. **LSTM** com arquitetura compacta: uma camada recorrente seguida de projeção linear $32 \rightarrow 1$. Entrada: janela de 288 amostras (24 h a 5 min) com 3 *features* (valor normalizado, $\sin$ e $\cos$ da hora-do-dia para codificar periodicidade). Treinada com 100 épocas e função de perda MSE; previsão recursiva passo-a-passo.

A escolha do modelo em produção é automatizada em quatro patamares: (i) modelo Holt-Winters previamente treinado e serializado, usado em modo de inferência; (ii) Holt-Winters *online*; (iii) SES em caso de dados insuficientes; (iv) repetição do último valor (*naive*). O *frontend* expõe ao utilizador qual o modelo em uso a cada momento — importante para honestidade epistémica: quando a série é demasiado curta a previsão é uma linha plana e o utilizador é informado disso.

Volume de dados e validação. O protótipo recolheu ~ 8 h contínuas de telemetria real (16/05/2026, 10:44–18:26; ~ 900 amostras a 30 s) — janela insuficiente para validar Holt-Winters ou treinar uma LSTM com generalização defensável. Para ilustrar o *pipeline* e permitir comparação preliminar, gerou-se histórico sintético

com 14 dias de sazonalidade diária realista (temperatura 20–26 °C com pico às ~15h, humidade invertida 40–65%, MQ-135 e ruído linearmente acoplados à ocupação seguindo um padrão típico de biblioteca). Numa janela de teste de 6 h, observou-se MAE de 0.22 °C (baseline horária), 0.28 °C (Holt-Winters) e 0.17 °C (LSTM) — Figura 4. Estes resultados ilustram o *pipeline* e não constituem evidência de desempenho em condições reais; o serviço recua para SES ou *naive* enquanto o histórico for insuficiente. A previsão de *ocupação* fica como trabalho futuro: depende de fatores externos (horários, exames, dia da semana) com baixa estacionariedade e exige meses a anos de histórico real.

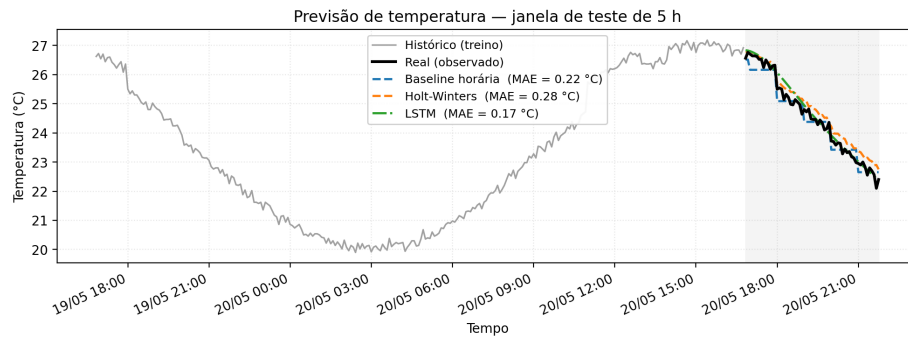


Figura 4. Comparação dos três modelos preditivos em dados sintéticos.

5 Modelo de Negócio

5.1 Proposta de Valor, Segmentação e Parcerias

A solução oferece três benefícios mensuráveis a uma instituição que gere espaços de estudo: redução do tempo médio de procura de um lugar livre (estimável em ~5 min por aluno em períodos de pico), melhoria da qualidade do estudo via monitorização do conforto ambiental — permitindo aos alunos escolher a sala com melhores condições e à instituição detetar e corrigir salas com deficiências de ventilação ou iluminação — e geração de dados estratégicos para decisões de gestão informadas (*e.g.* horários de pico, sazonalidade), em linha com a literatura de *smart campus* [1]. Identificámos três segmentos prioritários: *(i)* reitorias e serviços de documentação universitários, com ciclo de compra institucional alinhado a planos de modernização digital — a pertença académica do grupo torna a Universidade do Minho um piloto natural; *(ii)* redes de bibliotecas municipais e regionais, onde as métricas de conforto têm utilidade direta na conformidade com normas de qualidade do ar interior em edifícios públicos; e *(iii)* operadores de *coworking* e bibliotecas privadas, com maior disposição para pagar mas menor volume.

A viabilidade comercial de uma versão produtizada dependeria de parcerias **acadêmicas** (pilotos co-financiados com universidades), **tecnológicas** (integradores BMS para interoperabilidade com sistemas de gestão de edifícios existentes) e de **fabricao** (PCBs via JLCPCB/PCBWay e *enclosures* 3D). A arquitetura é generalizável a outros contextos de espaço partilhado com ocupação variável — transportes públicos, salas de espera hospitalares, espaços municipais — onde os dados de ocupação e conforto permitem igualmente otimização operacional.

5.2 Análise de Custos

A arquitetura admite três estratégias de implantação por sala (Tabela 1), distintas em dois eixos: (i) *origem da imagem*, câmara ESP32-CAM dedicada de baixo custo, câmara IP em rede interna do cliente, ou câmara MIPI/USB ligada a um *single-board computer* com GPU; e (ii) *localização da inferência*, *cloud* externa, servidor central na rede do cliente, ou *edge* na própria sala. Combinações puramente não-visuais, como WiFi-sensing ou PIR, foram descartadas por não distinguirem *seat hogging* [1]. O nó ambiental é comum a todos os cenários.

Tabela 1. Custo estimado por sala em três cenários. CAPEX em € de materiais (sem licenciamento software). OPEX em €/mês. Valores indicativos a preços 2025/26.

Componente	A protótipo	B centralizado	C edge
Nó ambiental	40 €	40 €	40 €
Aquisição visual	ESP32-CAM 10 €	0 €(B1) – 40 €(B2)	Jetson 240 €
Cabos / PCB / encl.	15 €	10–15 €	25 €
Servidor inferência	0 €	500 €/ No. de salas	0 €
CAPEX/sala	~65 €	~550–605 €/ No. de salas	~345 €
Firebase / <i>cloud</i>	4 €/mês	2–3 €/mês	1 €/mês
Energia + manutenção	2 €/mês	2–4 €/mês	3 €/mês
OPEX/sala/mês	~6 €	~4–7 €	~4 €
Imagens fora da sala	sim (<i>cloud</i>)	rede interna	não
Granularidade	por mesa	por mesa a por cadeira	por cadeira

Cenário A — protótipo atual. O sistema descrito na Secção 2 corresponde a este cenário: dois nós ESP32 em *breadboard* (~65 €/sala), servidor Python externo a correr a inferência sobre imagens transferidas via Firebase. A vocação é de *piloto de 1–2 salas*: prova-se o *pipeline* e gera-se histórico real para validar os modelos preditivos sem CAPEX significativo. A sua principal fraqueza é também o seu fundamento — a transmissão de imagens à *cloud* pública, mesmo que efêmera

—, o que o torna pouco apetecível para clientes maduros com políticas restritivas de saída de dados.

Cenário B — centralizado on-premise. As câmaras publicam o vídeo (RTSP/ONVIF) na rede interna do cliente, e um servidor central com GPU consome em paralelo os *streams* de todas as salas, executando o *pipeline* YOLO e publicando apenas os metadados na infraestrutura partilhada. As imagens nunca saem da rede interna e o custo do servidor (~ 500 €) amortiza-se por todas as salas servidas. Distinguem-se duas variantes pela origem da câmara: **B1** reaproveita o CCTV existente do edifício (apenas ~ 50 € de *hardware* por sala, dominado pelo nó ambiental), com duas contraindicações — a finalidade declarada no registo do EPD é “videovigilância de segurança” e estendê-la obriga a nova AIPD; e os ângulos típicos do CCTV (campo amplo, ligeiramente picado) degradam frequentemente a granularidade de por mesa para por zona. **B2** instala câmaras IP novas dedicadas à analítica (montagem cenital, sensor adequado, eventualmente *fisheye*), com ~ 95 € de *hardware* por sala mas controlando o ângulo — a granularidade pode recuperar até por cadeira e a base legal é mais limpa. Em campi com CCTV já universal nas salas de estudo, B1 é o ponto de entrada financeiramente óbvio; **B2** é a alternativa em edifícios sem CCTV adequado ou quando a finalidade do tratamento não suporta extensão.

Cenário C — edge AI dedicado por sala. Toda a inferência corre localmente num *single-board computer* (Jetson Orin Nano, Raspberry Pi 5 + Coral Edge TPU, ou equivalente) [1] com câmara MIPI/USB acoplada. A imagem nunca sai da sala: apenas os metadados são publicados na infraestrutura central via HTTPS, eliminando tanto o envio para *cloud* pública (como em A) como a circulação na LAN interna (como em B). É a escolha por excelência para clientes em que o quadro de proteção de dados é um *deal-breaker*. O custo unitário sobe (~ 345 €/sala, dominado pela Jetson) mas o OPEX cai; o nó ambiental pode ser integrado nos GPIO/I²C do próprio Jetson, reduzindo o BOM agregado para próximo dos 300 €.

Estratégia de rollout. Para uma instituição parceira sugere-se: (i) começar em A para piloto de 1–2 salas e geração de evidência para o EPD; (ii) migrar para B1 sempre que o CCTV institucional permitir o reaproveitamento das câmaras; (iii) recorrer a B2 nas salas em que o posicionamento do CCTV existente não dê granularidade suficiente; e (iv) manter C como *overlay* em salas com requisitos de privacidade elevados ou em zonas politicamente sensíveis (gabinetes, salas de pesquisa, espaços de leitura silenciosa).

5.3 Custos de Software e Licenciamento

A análise da Tabela 1 cobre *hardware* e infraestrutura, mas ignora deliberadamente o *software* — material quando a solução transita de protótipo académico a oferta comercial.

Modelo de inferência. O Ultralytics YOLOv11 está distribuído sob **AGPL-3.0** [2], que exige *open-source* de todo o código que se ligue ao modelo via rede — incompatível com SaaS fechado. Existem três caminhos para a fase comercial: (i) Ultralytics Enterprise License ($\sim 5\,000$ USD/ano para organização pequena); (ii) modelo alternativo com licença permissiva (YOLOX, YOLO-NAS, PP-YOLO, RT-DETR — todos Apache 2.0), ao custo de *fine-tuning* para ângulos cenitais; (iii) serviço terceiro com sub-licença (*e.g.* Roboflow Inference), que troca custo fixo por SaaS recorrente.

Cloud e TCO operacional. O nível gratuito *Spark* do Firebase acomoda o protótipo; em produção, o plano *Blaze* (*pay-as-you-go*) é o que sustenta o valor de Firebase já contabilizado na Tabela 1. Ativar a eliminação automática de imagens após inferência reduz esse custo abaixo de 0,50 €/mês/sala (os cenários B e C eliminam-no por não exportarem imagens). Acresce um custo *não* refletido na tabela: a *operação* de uma equipa SaaS pequena (1–2 engenheiros a $\sim 40\%$ FTE) ronda 50–80 k€/ano em Portugal — valor que ultrapassa o custo de *hardware* acima de ~ 50 salas e justifica automação de provisionamento e atualizações remotas de *firmware*.

5.4 Monetização e Riscos

Propomos um modelo B2B SaaS com três níveis: **Pilot** (*free*, 1 sala, 30 dias); **Campus Standard** (~ 35 – 50 €/sala/mês sem *hardware* incluído, mas com licenciamento e suporte); e **Campus Plus** (personalizado, com SSO/BMS/reservas, SLA 99,5%). *Setup fee* inicial dependente do cenário (~ 200 € em A, ~ 150 – 300 € em B consoante a variante, ~ 600 € em C). A margem estabiliza acima de 60% a partir das 200 salas, assumindo Enterprise License amortizada e suporte automatizado. Os principais riscos são quatro: *regulatório* (RGPD impõe obrigações pesadas a sistemas que capturem imagens, mesmo eliminadas pós-inferência — migração para *edge* é central para clientes avançados), *competitivo* (WiFi-sensing oferece dados de sala com instalação quase nula; a nossa diferenciação assenta no *seat hogging* e conforto, ambos exigindo sensores físicos), *licenciamento* (mudança nas condições da Enterprise License Ultralytics pode forçar migração para Apache 2.0 — risco mitigado por desacoplar o modelo da *interface* do detector) e *de adoção* (percepção de vigilância — ver inquérito, Secção 6.2).

6 Resultados e Análise Crítica

6.1 Funcionamento do Protótipo

A integração *end-to-end* foi validada com instalação dos dois nós em ambiente controlado, configurados para monitorizar uma mesa com quatro cadeiras. Os principais indicadores são apresentados na Tabela 2.

Tabela 2. Resultados experimentais do protótipo.

Indicador	Resultado observado
Intervalo de captura ESP32-CAM	15–20 s (UXGA saturava Wi-Fi a >60 s)
Tamanho médio do JPEG (SVGA, $q=12$)	50–75 kB
Latência de processamento (download $\rightarrow$ LED)	3–8 s (<i>download</i> + YOLO + escrita)
Descoberta automática de <i>layout</i>	convergência em 5 <i>frames</i> com sala vazia
Precisão de deteção de pessoa	<i>precision</i> > 95% (<i>conf</i> $\geq$ 0.40)
Deteção de <i>seat hogging</i>	detetado ao nível da mesa, apenas na ausência de pessoas; falha se objetos ficam ocultos sob a mesa no ângulo picado
Recall de pessoa (luz natural)	qualitativamente alto
Recall de pessoa (luminosidade noturna)	quedas perceptíveis, sobretudo em objetos escuros
Intervalo de leitura ambiental	30 s sem falhas em 24 h contínuas
Volume de dados ambientais reais	~ 900 amostras / ~ 8 h (16/05/2026)

6.2 Validação por Inquérito a Utilizadores

Para complementar a avaliação técnica foi distribuído um inquérito *online* à comunidade académica da UMinho com 9 itens fechados (escala Likert 1–5) e 2 perguntas abertas. Foram recolhidas $N=26$ respostas, maioritariamente de alunos com utilização semanal ou superior (16/26); a totalidade (26/26) já se tinha deslocado a uma sala de estudo e encontrado-a cheia (25 várias vezes, 1 raramente). A Tabela 3 resume os itens-chave.

Tabela 3. Resultados do inquérito a utilizadores ($N=26$). Escala 1–5.

Item	Média % ≥ 4	
P4. A solução é útil para a vida académica	4.58	96%
P5. A interface parece fácil de usar	4.46	96%
P6. Confiaria nos dados de ocupação apresentados	4.15	88%
P7. Confortável com captura de imagens (apagadas pós-inferência, sem reconhecimento facial)	3.96	73%
P8. Usaria a solução antes de se deslocar à sala	4.58	92%
P9. O semáforo à entrada pouparia deslocações	4.38	81%

Leitura. A proposta de valor é validada inequivocamente: utilidade (P4, 96%), usabilidade (P5, 96%) e intenção de uso (P8, 92%) reúnem concordância elevada e estável. A maior dispersão concentra-se em dois itens previsíveis: o conforto

com a captura de imagens (P7, média 3.96, com 27% das respostas neutras ou negativas) confirma que a percepção de vigilância existe *mesmo* quando se garante eliminação imediata e ausência de reconhecimento facial, reforçando a prioridade da migração para *edge* AI (cenário C); a utilidade do semáforo (P9, 81%) reúne 5 respostas neutras ou negativas, alinhadas com respostas abertas que questionam a sua mais-valia face ao *dashboard*. As respostas abertas convergem em três pontos: três inquiridos questionaram a utilidade do estado amarelo, sugerindo reservá-lo à planta detalhada; vários validaram a heurística *seat hogging* propondo limiar temporal (*e.g.* “alerta para mesas sem pessoas há mais de 15 min”); e as funcionalidades desejadas convergiram em notificação quando uma sala se liberta (3 menções), previsão de ocupação a 30–60 min (2 menções) e reserva de mesa (1 menção) — as duas primeiras já parcialmente implementadas.

6.3 Análise Crítica

Visão computacional. A inferência depende fortemente da iluminação — alta precisão diurna, mas quedas no *recall* em luminosidade noturna, sobretudo em objetos escuros. O *checkpoint* extra-large do YOLOv11 não cabe em *edge devices* típicos, inviabilizando inferência na própria ESP32-CAM sem regressão; a solução é o cenário C com Jetson. O ângulo picado nem sempre capta objetos sob a mesa, limitando a deteção de *seat hogging* a objetos visíveis; a granularidade ao lugar individual requer câmara em posição mais favorável à distinção de cadeiras individuais.

Sensores analógicos e MEMS. As limitações de precisão são consequência direta da escolha de sensores de baixo custo. O MQ-135 mede uma resposta agregada a múltiplos gases redutores e não isola CO₂, pelo que se apresenta como classe textual, nunca como ppm; a substituição pelo SCD30/SCD41 (NDIR) resolveria a leitura em ppm e tornaria as medições conformes com normas de qualidade do ar interior. Analogamente, a conversão para dB no microfone MEMS produz valores plausíveis (~35 dB em silêncio, ~70 dB em conversação) mas não substitui calibração com sonómetro Classe 2 (IEC 61672) — as classes textuais são confiáveis, o valor absoluto não. Ambas as substituições são recomendadas em cliente final com exigências de conformidade, ao custo de aumentar o BOM por sala.

Preditivo e privacidade. A janela de dados reais é insuficiente para validar os modelos com rigor — os resultados da Secção 4.2 ilustram o *pipeline* e não constituem evidência em condições reais. O serviço recua automaticamente para SES ou *naive* enquanto o histórico for insuficiente, evitando previsões falsamente precisas. Quanto à arquitetura, a separação em dois projetos Firebase concretizou o *privacy-by-design*, mas a transmissão de imagens à *cloud* permanece o ponto fraco. As credenciais ainda no *firmware* devem migrar para autenticação por token por dispositivo; a migração para o cenário B (*on-premise*) é o passo natural de escala e o C (*edge* AI) a evolução prioritária para clientes com EPD restritivo.

7 Conclusões e Recomendações

Demonstrou-se a viabilidade técnica da plataforma, com BOM por sala na ordem dos 65 € no cenário base. A integração *end-to-end* foi validada, da captura pela ESP32-CAM ao *dashboard* web multi-sala e à aplicação Android com *geofencing*, alertas configuráveis e catálogo. O *pipeline* de visão distingue presença humana de *seat hogging* sem rastreamento individual. O inquérito ($N=26$) confirmou utilidade percebida em 96% e intenção de uso em 92%; o ponto mais sensível foi o conforto com a captura de imagens (P7, 27% neutros ou negativos), reforçando a prioridade da migração para *edge* AI. A análise crítica delimita o sistema a um papel de *alerta qualitativo* e assume a comparação Holt-Winters/LSTM como preliminar, obtida sobre dados sintéticos por insuficiência de histórico real (~ 8 h).

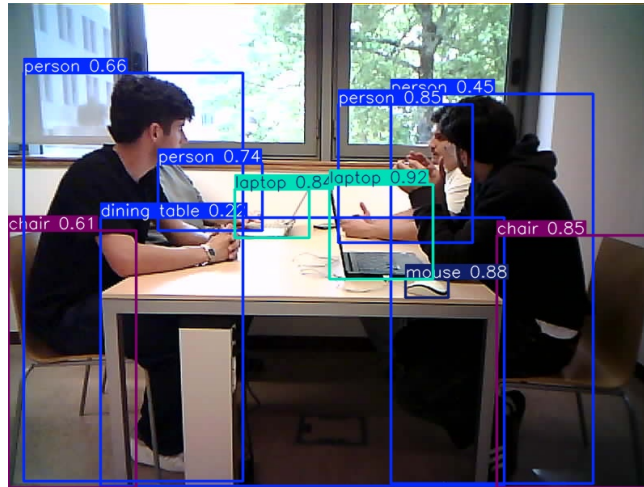
A plataforma é deliberadamente apresentada como um protótipo (cenário A) de uma arquitetura de referência que admite outros dois pontos de implantação — centralizado *on-premise* (B), com câmaras IP reaproveitadas do CCTV existente (B1) ou novas dedicadas (B2), e *edge* AI por sala (C). A separação entre o contrato de dados (API REST, *schema* JSON) e os blocos substituíveis (aquisição visual, local de inferência) permite que esta evolução seja incremental, sem descontinuar a UI ou os clientes existentes. A análise de custos inclui não apenas *hardware* mas também licenciamento de *software*, custos de *cloud* em produção e TCO operacional — elementos materiais para qualquer transição do protótipo a oferta comercial.

Recomendamos, por ordem de impacto: (i) recolha de histórico real prolongado para validar o componente preditivo, cuja precisão melhora monotonicamente com o volume acumulado; (ii) atribuição por cadeira com câmara em ângulo mais propício, devolvendo granularidade ao lugar individual; (iii) migrar inferência YOLO para um servidor central *on-premise* ou para *edge*, eliminando o envio de imagens para *cloud* pública; (iv) substituir MQ-135 por SCD30/SCD41 (NDIR) e LM393 por VEML7700, passando a medições conformes com normas EN; (v) decisão de licenciamento do modelo de inferência, pré-requisito de qualquer comercialização; (vi) endurecimento operacional e governança participada (autenticação por token por dispositivo, eliminação automática de imagens por defeito, sinalização à entrada e canal de *opt-out* com a Associação Académica); e (vii) estudo de viabilidade do cenário B1 com os Serviços de Documentação e o EPD da UMinho, dado o potencial de redução de CAPEX em campi já equipados com videovigilância moderna. A versão atual cumpre o enunciado e é base honesta para essas evoluções.

A Exemplo Qualitativo da Inferência Visual



(a) Imagem original capturada pela ESP32-CAM.



(b) Saída do YOLOv11x.

Figura 5. Inferência sobre uma mesa de 4 lugares com 4 ocupantes. Sujeitos consentiram a utilização da imagem para fins académicos. Visíveis: quatro deteções de pessoas (0.85, 0.74, 0.66, 0.45), dois *laptops* (0.82, 0.92), rato (0.88), mesa (0.22) e cadeiras (0.85, 0.61).

Glossário

- ADC** *Analog-to-Digital Converter*. Conversor analógico-digital, circuito que converte sinais analógicos contínuos em valores digitais discretos.
- AGPL** *Affero General Public License*. Licença de software livre que estende a GPL ao uso através de rede (SaaS).
- AIPD** Avaliação de Impacto sobre a Proteção de Dados. Análise obrigatória pelo RGPD para tratamentos com risco elevado.
- API** *Application Programming Interface*. Interface de comunicação entre sistemas de *software* que define como os componentes interagem entre si.
- ASHRAE** *American Society of Heating, Refrigerating and Air-Conditioning Engineers*. Organização que publica normas de conforto térmico (e.g. ASHRAE 55).
- B2B** *Business-to-Business*. Modelo comercial em que o cliente é outra empresa ou instituição.
- BMS** *Building Management Systems*. Sistemas de gestão centralizada de edifícios (HVAC, iluminação, segurança).
- BOM** *Bill of Materials*. Custo agregado de todos os componentes de *hardware* necessários para uma instalação.
- BSP** *Board Support Package*. Conjunto de *drivers* e ficheiros de configuração específicos de uma placa.
- CAPEX** *Capital Expenditure*. Despesa de capital referente ao investimento inicial em *hardware* e infraestrutura.
- CCTV** *Closed-Circuit Television*. Sistema de videovigilância em circuito fechado.
- CEN** *Comité Européen de Normalisation*. Organismo europeu de normalização.
- COCO** *Common Objects in Context. Dataset* padrão para deteção de objetos, com 80 classes.
- CPU** *Central Processing Unit*. Processador central.
- dB** *decibel*. Unidade logarítmica de medida de intensidade sonora.
- dBFS** *decibels relative to Full Scale*. Decibéis relativos à escala máxima de um conversor digital.
- EN** *European Norm*. Norma europeia harmonizada (e.g. EN 12464-1 para iluminação).
- EPD** Encarregado de Proteção de Dados. Função exigida pelo RGPD para entidades que processam dados pessoais em larga escala.
- FTE** *Full-Time Equivalent*. Equivalente a tempo integral, medida de carga de trabalho.
- GPIO** *General-Purpose Input/Output*. Pino de entrada ou saída programável em microcontroladores.
- GPU** *Graphics Processing Unit*. Unidade de processamento gráfico, usada para acelerar inferência de redes neurais.
- HTML** *HyperText Markup Language*. Linguagem de marcação para construção de páginas web.
- HTTPS** *HyperText Transfer Protocol Secure*. Protocolo HTTP cifrado por TLS.

- IEC** *International Electrotechnical Commission*. Comissão internacional que publica normas electrotécnicas (e.g. IEC 61672 para sonómetros).
- IoT** *Internet of Things*. Internet das coisas; rede de dispositivos físicos com sensores e conectividade.
- IoU** *Intersection over Union*. Métrica de sobreposição entre *bounding boxes* em deteção de objetos.
- I²S** *Inter-IC Sound*. Protocolo de comunicação digital para áudio entre circuitos integrados.
- JPEG** *Joint Photographic Experts Group*. Formato de compressão de imagem com perdas.
- JSON** *JavaScript Object Notation*. Formato textual de troca de dados estruturados.
- LAN** *Local Area Network*. Rede local.
- LED** *Light-Emitting Diode*. Díodo emissor de luz.
- LSTM** *Long Short-Term Memory*. Rede neuronal recorrente desenhada para captar dependências temporais longas.
- MAE** *Mean Absolute Error*. Erro absoluto médio, métrica de avaliação para previsão.
- MEMS** *Micro-Electro-Mechanical Systems*. Sistemas micro-eletromecânicos (e.g. microfones, acelerómetros, giroscópios).
- MIPI** *Mobile Industry Processor Interface*. Família de interfaces série para câmaras e ecrãs.
- MSE** *Mean Squared Error*. Erro quadrático médio, função de perda comum em regressão.
- NDIR** *Non-Dispersive Infrared*. Princípio de medição de gases por absorção de infravermelhos, padrão para CO₂.
- OMS** Organização Mundial de Saúde. Agência das Nações Unidas que publica recomendações sanitárias (e.g. limites de exposição ao ruído).
- ONVIF** *Open Network Video Interface Forum. Standard* de interoperabilidade entre câmaras IP de diferentes fabricantes.
- OPEX** *Operational Expenditure*. Despesa operacional recorrente, como energia, manutenção e serviços *cloud*.
- PCB** *Printed Circuit Board*. Placa de circuito impresso.
- PIR** *Passive Infrared*. Sensor de presença por deteção de radiação infravermelha.
- ppm** *parts per million*. Partes por milhão; unidade de concentração usada em qualidade do ar.
- PSRAM** *Pseudo-Static RAM*. Memória externa adicional usada por alguns SoC para alargar a RAM disponível.
- PWM** *Pulse Width Modulation*. Modulação por largura de impulso, usada para controlo de brilho de LEDs ou motores.
- REST** *REpresentational State Transfer*. Estilo arquitetural para APIs HTTP.
- RGPD** Regulamento Geral sobre a Proteção de Dados. Regulamento europeu (2016/679) que estabelece as regras para o tratamento de dados pessoais na União Europeia.

RMS *Root Mean Square*. Raiz quadrada da média dos quadrados; usado em sinais para medir amplitude.

RTSP *Real-Time Streaming Protocol*. Protocolo de transmissão de vídeo em tempo real, usado por câmaras IP.

SaaS *Software as a Service*. Modelo de distribuição de *software* por subscrição, geralmente em *cloud*.

SES Suavização Exponencial Simples. Método de previsão baseado em médias ponderadas exponencialmente decrescentes.

SLA *Service Level Agreement*. Acordo de nível de serviço, estabelece garantias contratuais (e.g. disponibilidade).

SoC *System on Chip*. Sistema integrado num único circuito (CPU, memória e periféricos).

SPL *Sound Pressure Level*. Nível de pressão sonora, medido em dB.

SSO *Single Sign-On*. Autenticação única que dá acesso a múltiplos serviços.

SVGA *Super Video Graphics Array*. Resolução de 800×600 pixéis.

TCO *Total Cost of Ownership*. Custo total de propriedade ao longo do ciclo de vida de uma solução.

TLS *Transport Layer Security*. Protocolo criptográfico que assegura comunicações em rede.

TPU *Tensor Processing Unit*. Acelerador dedicado a operações de redes neurais.

USB *Universal Serial Bus*. Padrão universal de ligação série entre dispositivos.

UX *User Experience*. Experiência do utilizador.

UXGA *Ultra eXtended Graphics Array*. Resolução de 1600×1200 pixéis.

YOLO *You Only Look Once*. Família de redes neuronais para deteção de objetos em tempo real.

Referências

1. Silva, L., Reis, P., Pinto, G., Gomes, P.: Sala de Estudo Inteligente: Levantamento do Estado da Arte sobre Deteção de Ocupação e Conforto Ambiental. Trabalho de Investigação, Sensorização e Ambiente, Universidade do Minho (2026)
2. Jocher, G., Chaurasia, A., Qiu, J.: Ultralytics YOLOv8/YOLOv11. <https://github.com/ultralytics/ultralytics> (2024). Licença AGPL-3.0; Enterprise License disponível em <https://www.ultralytics.com/license>
3. ASHRAE: ANSI/ASHRAE Standard 55-2023: Thermal Environmental Conditions for Human Occupancy. ASHRAE, Peachtree Corners (2023). <https://www.ashrae.org/technical-resources/bookstore/standard-55-thermal-environmental-conditions-for-human-occupancy>
4. CEN: EN 12464-1:2021 — Light and lighting — Lighting of work places — Part 1: Indoor work places. European Committee for Standardization, Brussels (2021).
5. World Health Organization, Regional Office for Europe: Environmental Noise Guidelines for the European Region. WHO, Copenhagen (2018). ISBN 978-92-890-5356-3. <https://www.who.int/europe/publications/i/item/9789289053563>
6. Google: Firebase Realtime Database and Storage Documentation. <https://firebase.google.com/docs> (2026)